WEB APPLICATION VULNERABILITY ASSESSMENT REPORT

- Target Name: manageengine.com
- Date: 23/05/2026
- Prepared By: karthick.R, Cybersecurity Intern
- Prepared For: Future Intern

DISCLAIMER & CONFIDENTIALITY

Confidentiality Statement:

This document contains confidential information regarding the security posture of the target application. It is intended solely for authorized personnel. Disclaimer: This assessment was conducted passively and ethically, utilizing non intrusive methods. The findings represent the security posture at the specific time of testing. Security is a continuous process, and this report does not guarantee complete immunity from future threats

EXECUTIVE SUMMARY:

- Heading: Executive Summary
- Text: "On 11/06/2026, a passive vulnerability assessment was conducted on manageengine.com. The objective was to identify security misconfigurations and exposed services without impacting the website's availability.
- The Verdict: "Overall, the application functions as intended, but several areas require immediate attention. The assessment identified 1 High-Risk, 3 Medium-Risk, and 1 Low-Risk vulnerabilities. The most critical issue is the exposure of outdated administrative services (SSH) to the public internet, which could allow unauthorized access. Other findings relate to missing modern web security configurations." Recommendation: "We recommend prioritizing the restriction of administrative ports, followed by updating server configurations to include modern security headers.

SCOPE & METHODOLOGY:

- Target Scope: http://www.manageengine.com
- Testing Methodology: Passive Reconnaissance, Open-Source Intelligence (OSINT), and Non Intrusive Scanning.
- Tools Utilized: Nmap, OWASP ZAP (Passive Mode), Browser Developer Tools.
- Exclusions: No active exploitation, brute-forcing, or Denial of Service (DoS) attacks were performed.

FINDINGS SUMMARY DASHBOARD

- Include a pie chart or a simple table showing the count of vulnerabilities by severity.
- 🔴 High: 1
- 🟠 Medium: 3
- 🟡 Low: 1
- Here the chart, provide a list of the findings:
  a. Exposed Outdated SSH Service (High)
  b. Information Disclosure via Server Headers (Medium)
  c. Insecure Cookie Configuration (Medium)
  d. Absence of Anti-CSRF Tokens (Medium)
  e. Missing HTTP Security Headers (Low)

FINDING #1

Exposed & Outdated Administrative Service (SSH)

- Severity: High Description: Port 22 (SSH) is currently open to the public internet. The service running is identified as OpenSSH 6.6.1p1, which was released in 2014.
- Business Impact: SSH provides direct command line access to the server. Leaving it exposed allows attackers password-guessing to perform automated attacks. Because the software is severely outdated, it may also be vulnerable to known public exploits, potentially leading to a total server compromise.
- Proof of Concept (Evidence): (Insert your screenshot of the Nmap scan here showing port 22 open and the version)
- Remediation / Fix:
- a.Configure the server firewall (e.g., UFW or AWS Security Groups) to block Port 22 from the public internet, allowing access only from trusted corporate IP addresses or a VPN. b.Disable password authentication and require cryptographic SSH Keys for login. c.Update OpenSSH to the latest stable release.

FINDING 2

Information Disclosure via Server Headers

- Severity: Medium Description: During passive HTTP inspection, it was discovered that the web server includes the Server header in its HTTP responses, specifically broadcasting the exact software and OS version: Apache/2.4.7 (Ubuntu).
- Business Impact: Broadcasting exact software versions provides attackers with a precise blueprint of the server's architecture. Instead of spending time probing for vulnerabilities, an attacker can simply search vulnerability databases for known exploits specific to Apache 2.4.7, significantly reducing the time and effort required to launch a successful attack.
- Remediation: Configure the Apache web server to obscure or completely suppress the Server header. In Apache, this is done by setting ServerTokens Prod and ServerSignature Off in the configuration file.

FINDING 3

Insecure Cookie Configuration

- Severity: Medium Description: The web application issues sensitive session cookies to the user's browser without utilizing the HttpOnly security flag.
- Business Impact: The HttpOnly flag prevents client-side scripts from accessing cookies. Without this flag, if a hacker successfully finds a Cross-Site Scripting (XSS) vulnerability anywhere on the site, they can use malicious JavaScript to steal the user's cookies. This allows the attacker to fully hijack the user's logged-in session without needing their username or password.
- Remediation: Modify the application framework or web server configuration to automatically append the HttpOnly and Secure attributes to all session cookies before they are sent to the client browser.

FINDING 4

Absence of Anti-CSRF Tokens

- Severity: Medium Description: Automated passive scanning via OWASP ZAP identified that HTML forms within the application do not utilize Cross-Site Request Forgery (CSRF) protection tokens.
- Business Impact: Without Anti-CSRF tokens, the application cannot distinguish between a legitimate user request and a forged request. An attacker could trick an authenticated user into clicking a link on a malicious third-party website, which silently submits a state changing action (like changing an account email or password) on the vulnerable site on the user's behalf.
- Remediation: Implement the synchronizer token pattern. Ensure that all state-changing requests (POST, PUT, DELETE) require a unique, cryptographically secure, and session-specific Anti-CSRF token that the server validates before processing the request.

FINDING 5

Missing HTTP Security Headers

- Severity: Low
- Description: The web application fails to implement modern HTTP security headers, notably X-Frame-Options, X-Content-Type Options, and Content-Security-Policy (CSP).
- Business Impact: Modern web browsers have built-in defenses against attacks, but they require the server to activate them via headers. Without X-Frame-Options, the website can be invisibly embedded inside a malicious site, facilitating "Clickjacking" attacks where users are tricked into clicking hidden buttons. The absence of a CSP makes the application much more vulnerable to XSS attacks.
- Remediation: Update the web server configuration to include robust security headers. At a minimum, implement X-Frame Options: DENY (or SAMEORIGIN) and X Content-Type-Options: nosniff. It is also highly recommended to develop and implement a strict Content-Security-Policy.

CONCLUSION

"Securing a web application is an ongoing process. Implementing the remediations outlined in this report will significantly reduce the attack surface of manageengine.com. We recommend scheduling a follow-up assessment after the remediation phase is complete to verify the fixes."